# 自研 106M 大语言模型数据侧能力迭代研究报告

MiMoLite · 无蒸馏规则化数据工程 · 8 版实验全记录

## 摘要

本项目从零自研 106M 参数代码生成大语言模型（MiMoLite），在“不蒸馏、不改模型结构、评测为唯一准绳”的硬约束下，通过规则化模板数据工程将代码评测通过率从基座 0% 提升至 pass@1 29%、pass@8 57%、pass@16 59%。迭代全程保持同训练量（330 步/10 epochs）只换数据，形成干净的对照矩阵。本文完整记录 8 版数据实验、四个关键根因修复、两次负优化实验的归因以及顽固区能力边界，全部命令与档案可复现、可核验。

## 目录

（目录：请在 Word 中右键选择“更新域”以刷新）

# 一、项目概述

## 1.1 研究目标

本项目的长期目标是验证一个命题：在不使用教师模型蒸馏、不扩大模型规模的前提下，仅靠规则化数据工程能把一个小模型（106M）的代码生成能力推到什么程度。当前阶段以自建 100 题代码评测集为唯一准绳，通过可复现的实验矩阵寻找数据侧的帕累托最优配置。

## 1.2 硬件与模型

训练与评测均在本机完成：NVIDIA RTX 5070 8GB，Windows 环境，PyTorch。模型为自研 106.1M 参数解码器，结构参数为 8 层、隐藏维度 960、15 注意力头、前馈维度 2560、窗口 128，启用 MTP 辅助（权重 0.3）。基座（a001_B1_v2）预训练 4500 步，中英混合语料，中文占比 0.75，学习率 1.5e-3。

基座对照评测结果：100 题全挂，通过率 0.0%。这排除了“基座已具备代码能力”的可能，后续全部提升归因于数据侧迭代。

## 1.3 核心约束

实验遵守三条硬约束：其一，评测集与训练集严格无泄漏（评测题面、函数名、测试用例均不进入训练数据）；其二，评测口径固定（同一 100 题、同一采样参数、同一判定逻辑），任何一次通过率的变动只由数据变更引起；其三，训练量同构（全部 SFT 版本 10 epochs、有效 batch 32，只换数据不换训练配置），保证对照干净。

## 1.4 与既有工作的定位

主流代码模型的数据策略可分为四类。其一，大规模语料混合：DeepSeek-Coder 以 87% 源码、10% 英文代码语料与 3% 中文语料预训练，并用 GRPO 做代码偏好对齐（Guo et al., arXiv:2401.14196）。其二，教师合成：phi-1 用 GPT-4 生成的教科书质量数据训练 1.3B 模型（Gunasekar et al., arXiv:2306.11644）。其三，自举：STaR 让模型用自身正确推理微调（Zelikman et al., arXiv:2203.14465）。其四，自生成指令：Self-Instruct 用模型自产指令扩展微调集（Wang et al., arXiv:2212.10560）。

上述路线的共同隐含假设是存在强教师、或数据规模与算力足够。本研究占据的约束组合——106M 参数、禁止教师蒸馏、600 条规则化合成预算、8GB 单卡——与 MiMo-V2.5 等百亿级模型后训练（48T token）相比规模相差 5 个数量级，但固定预算下的样本-函数分配问题在小模型上反而更尖锐：预算守恒律与函数名竞争律直接决定能力边界（见第五、六章）。

# 二、评测体系与无泄漏设计

## 2.1 评测集构成

评测集为 100 道代码生成题，中文 83 题、英文 17 题，覆盖 16 个主题簇。题目要求模型输出完整可运行代码（含必要的类定义），判定为子进程执行断言全部通过。主题分布如表 2 所示。

表 2 评测集主题构成（100 题）

| 主题簇 | 题数 | 主题簇 | 题数 |
|---|---|---|---|
| 递归基础 | 17 | 双指针 | 5 |
| 哈希表计数 | 14 | 位运算基础 | 4 |
| 字符串处理 | 9 | array traversal | 4 |
| 数组遍历与统计 | 9 | 贪心 | 4 |
| 排序与比较 | 8 | bitwise basics | 3 |
| 简单数学计算 | 8 | 链表基础操作 | 3 |
| hashmap counting | 6 | simple math / 其他 | 5 |

## 2.2 评测协议

评测脚本 eval_code.py 固定采样参数：temperature 0.2、top_k 50、top_p 0.9、重复惩罚 1.1、最大新 token 512、随机种子 2026、batch 4、每题超时 20 秒、bf16 推理。pass@1 为单次采样，pass@8 与 pass@16 为多次采样任一命中即过。评测沙箱不注入任何工具函数或类——模型必须自己写出 ListNode 等必要定义，这一设计与真实评测更接近。

## 2.3 无泄漏构建

评测集构建时记录了全部 100 题的原始行号（eval_src_lines）。训练数据构建脚本 build_train_v2.py 在合并前按行号剔除这些题目，并做二次泄漏校验（题面、函数名、

代码片段三重检查），最终训练集泄漏检查结果为 0。早期实验曾因漏隔离评测行造成通过率虚高（1%/4%），该问题在正式迭代前已彻底修复，v5 起全部为无泄漏结果。

# 三、数据合成方案：不蒸馏的核心

## 3.1 模板变体设计

数据侧迭代的变量是 600 条规则化模板合成样本。模板库按主题组织，每条变体包含中英双语句面、参考实现、推理过程和 3 组随机测试断言。变体设计以“函数名对齐评测接口”为原则：评测高频函数名逐一建立对应变体，变体数从初版 35 个扩至 75 个，覆盖评测 16 个主题簇中的全部核心函数。样本按评测频次加权抽样：15 个高频函数权重 16，其余变体权重 6（单函数单语言上限 16/6 条）。

## 3.2 程序化生成与自验证

合成不使用教师模型。每条样本的测试断言在子进程中实际执行参考实现，验证通过才写入数据集；生成时随机扰动输入与边界（空输入、负值、单元素、重复元素等），保证测试多样性。600 条模板样本 + 510 条早期固定程序化样本构成 1110 条 SFT 训练集（v10 为 1154 条，见第五章）。

## 3.3 关键根因修复

迭代中定位并修复了四个影响数据质量的根因问题，均为“模板与评测接口不一致”类缺陷。修复清单如表 4 所示。

表 4 关键根因修复清单

| 编号 | 缺陷 | 表现 | 修复 | 验证 |
|---|---|---|---|---|
| R1 | _case() 四个分支（factorial/power/count_digits/count_set_bits）输入与期望使用不同随机数 | 同一样本输入与答案不匹配，模型学到错误映射 | 统一使用同一随机数生成输入并推导期望 | factorial 12 题通过 9 题（修复前接近 0） |
| R2 | process_string 模板逻辑与评测题面不符（模板转小写+过滤，评测转大写+去空格） | 该函数评测全挂 | 重写模板逻辑与题面，对齐评测语义 | pass@8 该函数 4/4 全过 |

| 编号 | 缺陷 | 表现 | 修复 | 验证 |
|---|---|---|---|---|
| R3 | 链表 gen_tests 用 ListNode(3,7,1) 扁平构造，不符合构造器语义 | 链表样本自验证失败率高 | 改为链式 ListNode(v, 外层表达式) 尾部构建，保持链头 =vals[0] | 链表样本可稳定生成 |
| R4 | 评测沙箱不注入 ListNode，模型必须自写类定义 | 模型输出 NameError（链表题全挂） | 模板中完整教学 ListNode 类定义 + 反转实现 | pass@8 链表 3 题可命中 |

# 四、实验矩阵与结果

## 4.1 早期存档（v1–v4）

v1–v4 为泄漏隔离修复前的早期存档：训练数据包含评测源行，评测结果虚高（旧泄漏档 pass@1 1%、pass@8 4%），仅作为流程记录保留，不参与正式对照。v5 起全部为无泄漏同口径结果。

## 4.2 正式同口径迭代（v5–v8）

v5–v8 四次迭代保持完全相同的训练量（1110 条 × 10 epochs = 330 步）只换数据，结果如表 1 所示。v5 为初版模板（35 变体、高频函数重复轰炸）；v6 扩至 75 变体均匀分配（多样性有效，pass@8 即达 v5 的 pass@16 水平）；v7 引入按评测频次加权抽样（高频权重 16）；v8 在 v7 基础上修复 process_string 模板逻辑，pass@8 升至 57%，为当前最佳权重档。

表 1 SFT 正式迭代实验矩阵（同训练量，只换数据）

| 版本 | 数据构成（模板部分） | pass@1 | pass@8 | pass@16 | 结论 |
|---|---|---|---|---|---|
| sft_v5 | 35 变体，高频重复轰炸 | 27% | 51% | 53% | 旧基线 |
| sft_v6 | 75 变体，均匀分配 | 25% | 53% | — | 多样性有效 |
| sft_v7 | v2 模板 + 加权抽样 (hot=16) | 30% | 53% | 58% | 加权有效 |

| 版本 | 数据构成（模板部分） | pass@1 | pass@8 | pass@16 | 结论 |
|---|---|---|---|---|---|
| sft_v8 | v7 + 修复 process_string 逻辑 | 29% | 57% | 59% | 当前最佳 |

## 4.3 关键结论

三条可复现的结论：第一，数据多样性直接提升采样上限——v6 的 pass@8 就追平 v5 的 pass@16，说明重复轰炸高频函数不如扩大变体覆盖；第二，加权抽样在多样性基础上把预算倾斜给评测高频函数，pass@1 提升最明显（v7 达 30%）；第三，模板与评测接口的对齐（R2）是 pass@8 从 53% 跳到 57% 的直接原因——数据质量缺陷修复的收益大于继续加样本。

## 4.4 外部基准锚点：内分布与外分布的泛化鸿沟

将当前最佳权重 sft_v8 在标准代码基准上做外分布锚点对照，结果见表 4-3。HumanEval（164 题）与 MBPP（257 题）均为英文函数级代码生成基准，与训练分布（中文指令、自建题面）在语言、题面风格与函数名集合上完全异源。两基准 pass@1 均为 0；pass@8 多采样后仍为 0，与内分布（自建 100 题）29/57 形成鲜明对比——数据侧迭代把能力绑定在了训练分布的题面-函数名组合上，尚未形成跨分布泛化。该对照同时披露适配口径：HumanEval 原版 docstring 续写与训练分布错配，评测采用指令式适配（追加“请实现函数”并运行时剔除中文），判定经逐例诊断确认真实，非评测误杀。

表 4-3 内外分布对照（sft_v8，同采样口径）

| 评测集 | 题量 | pass@1 | pass@8 |
|---|---|---|---|
| HumanEval（英文标准） | 164 | 0/164 (0%) | 0/164 (0%) |
| MBPP（英文标准） | 257 | 0/257 (0%) | 0/257 (0%) |
| 自建 100 题（内分布） | 100 | 29/100 (29%) | 57/100 (57%) |

### 4.5 多 seed 稳健性：结论是否依赖单次数据生成

自建数据链（模板合成→去重合并→SFT 格式）在同一配置下以不同随机种子重复三次，验证 29/57 是否为单点噪声（见表 4-4）。同训练量、同评测集、同采样口径，唯一变量为数据生成种子。

表 4-4 三 seed 稳健性对照（10 epochs，同口径）

| 版本 | 题量 | pass@1 | pass@8 |
|---|---|---|---|
| sft_v8（seed 7，v8 基线） | 100 | 29/100 (29%) | 57/100 (57%) |
| sft_v8_seed8 | 100 | 31/100 (31%) | 58/100 (58%) |
| sft_v8_seed9 | 100 | 24/100 (24%) | 52/100 (52%) |

## 五、负优化实验与归因

### 5.1 v9：删干扰变体 + 极端强化

为攻坚顽固区（merge/链表/count_words 共 9 题 pass@1 全挂），v9 删除 4 个与评测函数名近似的非评测变体（count_vowels/fibonacci/array_sum/count_evens），并将 3 个目标函数权重提到 32。结果 pass@1 从 29% 暴跌至 20%（-9pt）。逐题对比显示：v9 仅新过 2 题，回退 11 题，损失集中在 count_characters（8→6）、count_ones、count_elements 等真实评测高频函数——强化 3 个函数直接挤占了它们的训练预算。

### 5.2 v10：确定性配额 + 扩大总量

v10 修正 v9 的预算失衡：模板总量从 600 增至 720（实际 644），3 个目标函数各配额 40 条，其他高频函数保住 16 条份额。结果 pass@1 28%、pass@8 51%，仍低于 v8（29%/57%）。归因：总预算守恒——即使总量增加，40 条同质样本让模型对目标函数过度拟合；同时低样本函数的采样命中率下降（count_elements、count_ways 等回退），且目标函数 pass@1 依然 0 分。

### 5.3 归因：预算守恒与函数名竞争

两次负优化共同验证了一个数据侧铁律：在有限预算内强化 A 必然削弱 B，且被强化函数若本身存在能力瓶颈（函数名竞争、类定义能力），加样本量不会转化为通

过率。v8 的“600 条 / 高频权重 16 / 其余 6”配置即为数据侧局部最优，后续攻坚不应再走“加样本量”路线。

# 六、顽固区诊断与能力边界

## 6.1 评测设计死题：count_chars

评测中 count_chars 4 题与 count_characters 8 题语义完全同质（均为“统计字符次数 + 返回字典”），题面措辞无任何可区分信号，且训练数据中 count_characters 变体占比显著更高，模型只能学好一个名字（保大弃小）。采样维度复核显示 count_chars 4 题 pass@8 命中 1/4，并非完全死题——模型对同名函数已有部分能力，瓶颈在名字竞争与样本占比。

## 6.2 函数名竞争：merge / 链表 / count_words

三组共 9 题 pass@1 长期为 0，加样本量（16→40 条）无效。根因有二：一是函数名竞争——模型对相近语义题面（词频统计、合并数组）输出更熟悉的无关函数名（count_vowels/count_ways 等）；二是 106M 能力上限——链表题要求模型自写 ListNode 类并实现反转，小模型难以稳定生成完整类定义。二者均非数据量问题。

## 6.3 采样维度的部分命中

按函数名聚合 100 题（51 个函数）后，顽固区的采样命中全景比定性描述更清晰：13 个函数 pass@1 全挂但 pass@8 有命中（见表 6），其中 process_string 4/4、fib 4/4、sum_array 2/2、convert_to_ascii 2/2、count_set_bits 2/2 全部多采样命中——说明这些函数的能力已通过数据工程灌入权重，pass@1 仅受单采样确定性限制，模型真实能力被 pass@1 低估。其余双零死题（30 个函数）中，merge 系 5 题与大量单题稀有函数（sum_list/sum_even/flip_bits 等）受样本不足与函数名竞争共同制约，是评测集版本迭代的主要对象。

表 6 多采样命中函数清单（pass@1 = 0 但 pass@8 > 0，v8）

| 函数 | 评测题数 | pass@8 命中 |
|---|---|---|
| count_chars | 4 | 1 |
| process_string | 4 | 4 |
| fib | 4 | 4 |
| reverse_linked_list | 3 | 1 |

| 函数 | 评测题数 | pass@8 命中 |
|---|---|---|
| count_words | 3 | 1 |
| sum_array | 2 | 2 |
| sum_even_numbers | 2 | 1 |
| convert_to_ascii | 2 | 2 |
| count_set_bits | 2 | 2 |
| multiply | 1 | 1 |
| count_even_numbers | 1 | 1 |
| count_even | 1 | 1 |
| flip_bit | 1 | 1 |

表 5 顽固区诊断汇总

| 函数 | 评测题数 | pass@1 | 根因 | 处置 |
|---|---|---|---|---|
| count_chars | 4 | 0/4 | 与 count_characters 题面同质，评测无区分信号 | 部分命中（p8 1/4），保大弃小 |
| merge（三函数名） | 3 | 0/3 | 函数名竞争 + 生成不稳定 | 采样维度部分命中 |
| reverse_linked_list | 3 | 0/3 | 106M 需自写 ListNode 类 + 反转 | pass@8 3/3 命中 |
| count_words | 3 | 0/3 | 英文题面 + 函数名竞争（输出 count_vowels/count_ways） | 采样维度部分命中 |

# 七、复现指南

## 7.1 数据链

全部数据操作可通过以下命令链复现（PowerShell，Python 解释器为项目唯一 torch 环境）。每次重训前必须重新执行 build_train_v2.py 与 prepare_sft.py，保证训练集与模板版本一致。

表 7 复现命令链

| 步骤 | 命令 | 产出 |
|---|---|---|
| 生成模板样本 | synth_code_tpl.py --n 600 --seed 7 | data/synth_code_tpl.jsonl（600 条） |
| 合并无泄漏训练集 | build_train_v2.py | data/synth_code_v2.jsonl（1110 条，泄漏 0） |
| 构建 SFT 格式 | prepare_sft.py --raw data/synth_code_v2.jsonl --min 1 | data/synth_code_sft.jsonl |

| 步骤 | 命令 | 产出 |
| --- | --- | --- |
| 训练（10 epochs） | sft.py --ckpt runs/a001_B1_v2/final.pt --data data/synth_code_sft.jsonl --run-name sft_vX --epochs 10 --val-n 60 | runs/sft_vX/final.pt（330 步） |
| 评测 pass@1 | eval_code.py --ckpt runs/sft_vX/final.pt --out logs/eval_code_sftvX.json | logs/eval_code_sftvX.json |
| 评测 pass@8/16 | eval_code.py ... --samples 8/16 --out logs/eval_code_sftvX_pK.json | 采样命中档案 |

### 7.2 档案清单

权重：当前最佳 runs/sft_v8/final.pt（29/57/59）；基座 runs/a001_B1_v2/final.pt；实验档 runs/sft_v5~v10/final.pt。评测档案：logs/eval_code_sftv5*.json ~ eval_code_sftv10*.json（含 p8/p16 各档）。分析脚本：compare_v5_v6.py（逐题对比，改两行路径即可复用）、diag_zero.py/show_zero.py（顽固区诊断）、stats_fns.py/stats_topics.py（频次与主题统计）。GRPO 训练侧资产：grpo_code.py（代码沙箱奖励版）、select_grpo_data.py/filter_hard.py（OOD 题筛选）、data/grpo_train.jsonl 与 grpo_hard.jsonl、runs/grpo_code_v1~v4（四轮实验档案）

## 八、GRPO 训练侧实验（负结果）

### 8.1 动机与设计

### 8.2 四轮实验矩阵

### 8.3 归因与结论

为突破 SFT 数据上限、攻坚 pass@1 与顽固区，实现了 grpo_code.py：以代码沙箱可验证奖励为信号（断言通过 +1.0、无代码块 -0.5），policy + 冻结参考模型做 KL 锚定，组内相对优势更新；从当前最佳 sft_v8 权重初始化。过程中修复了早期 grpo.py 骨架的标量梯度缺陷（原版无法反向传播）。

四轮实验均在评测集 32 题上以低温采样（temp 0.2）评估，RL 前基线 pass@1 = 56.2%。结果矩阵如表 8 所示：模板题对 v8 过拟合导致无学习信号；筛选题信号不足；最难题上模型 rollout 全错，GRPO 退化为惩罚一切，评测跌至 43.8%。

**表 8 GRPO 四轮实验矩阵（评测 32 题 pass@1，RL 前 56.2%）**

| 轮次 | 训练数据 | 关键参数 | 终局评测 | 结论 |
|---|---|---|---|---|
| v1 | 模板题（v8 全会） | kl 0.01 / lr 1e-5 / temp 1.0 | 50.0% | 策略漂移，破坏基线 |
| v2 | 模板题 | kl 0.1 / lr 3e-6 / temp 0.7 | 56.2% | 无信号（rollout 全过） |
| v3 | 筛选题 264（通过率≤0.5） | 同上 | 56.2% | 信号不足（96 rollout） |
| v4 | 最难题 215（通过率=0） | 同上 + 60 组 | 43.8% | 惩罚一切，破坏能力 |

归因有三：其一，能力上限——106M 在最难题上连接近正确的 rollout 都产生不了，RL 缺少可强化的正样本；其二，连坐效应——惩罚失败代码的 token 会连带削弱与成功代码共享的语法与函数名模式；其三，规模差距——MiMo-7B 等成功案例依赖 13 万道可验证题的采样池与更大模型容量，8GB 显存限制了 group-size 与训练组数。

结论：在 106M + 8GB 规模下小规模 GRPO 暂不可行，先搁置。资产全部保留（grpo_code.py、OOD 题筛选脚本、data/grpo_train.jsonl 与 grpo_hard.jsonl、runs/grpo_code_v1~v4 四轮档案），模型规模升级后可复用。

## 九、结论与下一步

本研究以 8 版同口径实验证明：106M 模型的代码能力可以通过纯数据工程从 0% 提升至 pass@1 29%、pass@8 57%、pass@16 59%，且提升路径可复现、可归因。数据侧局部最优已收敛（v8 配置），两次负优化实验明确了“预算守恒”与“函数名竞争”两条边界。

下一步方向：其一，数据侧残余增量与评测集版本迭代（顽固区已归因于评测设计缺陷，可推动评测集完善）；其二，将本报告的实验矩阵与负结果（含第八章 GRPO 验证）作为研究叙事交付，形成敲门砖材料。GRPO 训练侧已实证在 106M 规模暂不可行，待模型规模升级后重启。